

SOLID Principles in C++: A Practical Demonstration

Introduction

Imagine you're building with blocks. SOLID principles are like rules for building with blocks that make your structure strong, easy to change, and fun to work with. If you follow these rules, your code won't fall apart when you want to add new features or fix bugs.

SOLID is an acronym for five design principles that help you write better code. Think of it like cooking: each principle is an ingredient that makes your code "taste" better (be more maintainable and flexible).

The principles are:

- **Single Responsibility Principle (SRP)** - One job per class
- **Open-Closed Principle (OCP)** - Open for extension, closed for modification
- **Liskov Substitution Principle (LSP)** - Subtypes should work like their parents
- **Interface Segregation Principle (ISP)** - Don't force classes to use methods they don't need
- **Dependency Inversion Principle (DIP)** - Depend on ideas, not specific things

We'll use a notification system (like sending emails, SMS, and push notifications) to show how these principles work.

The Problem

We need to build a notification system that can send notifications via email, SMS, and push notifications. The system should be extensible, maintainable, and follow SOLID principles.

Initial Design (Violating SOLID Principles)

```
#include <iostream>
#include <string>
#include <vector>

// Bad design - violates all SOLID principles
class NotificationService {
public:
    void sendNotification(const std::string& type, const std::string& message, const std::string& recipient) {
        if (type == "email") {
            // Email sending logic
            std::cout << "Sending email to " << recipient << ": " << message << std::endl;
            // Simulate email validation, SMTP connection, etc.
        } else if (type == "sms") {
            // SMS sending logic
            std::cout << "Sending SMS to " << recipient << ": " << message << std::endl;
            // Simulate SMS gateway connection, character limit checking, etc.
        } else if (type == "push") {
            // Push notification logic
            std::cout << "Sending push notification to " << recipient << ": " << message << std::endl;
            // Simulate device token validation, APNs/FCM connection, etc.
        }
    }
};

int main() {
    NotificationService service;
    service.sendNotification("email", "Welcome!", "user@example.com");
    service.sendNotification("sms", "Your code is 1234", "+1234567890");
    service.sendNotification("push", "New message", "device_token_123");
    return 0;
}
```

This design violates SOLID principles because:

- **SRP:** The class handles multiple responsibilities (email, SMS, push)
- **OCP:** Adding a new notification type requires modifying the class
- **LSP:** Not applicable yet
- **ISP:** Clients are forced to depend on methods they don't use
- **DIP:** High-level module depends on low-level implementation details

Refactored Design Following SOLID Principles

Step 1: Single Responsibility Principle (SRP) - "One Job Per Class"

Imagine: You're a chef. Would you cook, serve food, AND clean the kitchen all at once? No! You'd have separate people for cooking, serving, and cleaning. That's SRP - each class should do only ONE thing well.

The Problem (Bad Code):

```
class BadChef {
public:
    void cook() { /* cooking logic */ }
    void serve() { /* serving logic */ }
    void clean() { /* cleaning logic */ }
};
```

This class does too many things! If you change how cleaning works, you might break cooking accidentally.

The Solution (Good Code):

```
class Cook {
public:
    void cook() { /* only cooking */ }
};

class Server {
public:
    void serve() { /* only serving */ }
};

class Cleaner {
public:
    void clean() { /* only cleaning */ }
};
```

In Our Notification Example: Each notification sender class does ONLY one thing - sends one type of notification.

```

#include <iostream>
#include <string>

// Interface (like a blueprint) for notification sending
class INotificationSender {
public:
    virtual void send(const std::string& message, const std::string& recipient) = 0;
    virtual ~INotificationSender() = default;
};

// Email sender - ONLY sends emails, nothing else!
class EmailNotificationSender : public INotificationSender {
public:
    void send(const std::string& message, const std::string& recipient) override {
        // Email-specific logic only
        validateEmail(recipient);
        std::cout << "Sending email to " << recipient << ": " << message << std::endl;
        // Connect to email server, authenticate, send...
    }

private:
    void validateEmail(const std::string& email) {
        // Email validation logic
        if (email.find('@') == std::string::npos) {
            throw std::invalid_argument("Invalid email address");
        }
    }
};

// SMS sender - ONLY sends SMS, nothing else!
class SmsNotificationSender : public INotificationSender {
public:
    void send(const std::string& message, const std::string& recipient) override {
        // SMS-specific logic only
        validatePhoneNumber(recipient);
        checkMessageLength(message);
        std::cout << "Sending SMS to " << recipient << ": " << message << std::endl;
        // Connect to SMS gateway, send message...
    }

private:
    void validatePhoneNumber(const std::string& phone) {
        // Phone validation logic
        if (phone.empty() || phone[0] != '+') {
            throw std::invalid_argument("Invalid phone number");
        }
    }

    void checkMessageLength(const std::string& message) {
        if (message.length() > 160) {
            throw std::length_error("SMS message too long");
        }
    }
};

// Push sender - ONLY sends push notifications, nothing else!
class PushNotificationSender : public INotificationSender {
public:
    void send(const std::string& message, const std::string& recipient) override {
        // Push-specific logic only
        validateDeviceToken(recipient);
        std::cout << "Sending push notification to " << recipient << ": " << message << std::endl;
        // Connect to push service, send notification...
    }
};

```

```
private:
    void validateDeviceToken(const std::string& token) {
        // Token validation logic
        if (token.length() < 10) {
            throw std::invalid_argument("Invalid device token");
        }
    }
};
```

Why SRP is Important:

- **Easy to change:** Want to improve email sending? Only touch EmailNotificationSender
- **Easy to test:** Test each class separately
- **Less bugs:** Changes in one area don't break others
- **Reusable:** Use EmailNotificationSender in other projects

Step 2: Open-Closed Principle (OCP) - "Open for Extension, Closed for Modification"

****Imagine:**** You have a toy box. Instead of breaking it open to add new toys, you want to design it so you can just add toys through

****The Problem (Bad Code):****

```
```cpp
class ToyBox {
public:
 void addToy(std::string toyType) {
 if (toyType == "car") {
 // add car logic
 } else if (toyType == "doll") {
 // add doll logic
 }
 // To add a new toy, you must MODIFY this class!
 }
};
```

#### The Solution (Good Code):

```
class Toy {
public:
 virtual void play() = 0;
};

class Car : public Toy {
public:
 void play() override { /* car playing */ }
};

class Doll : public Toy {
public:
 void play() override { /* doll playing */ }
};

class ToyBox {
private:
 std::vector<std::unique_ptr<Toy>> toys;
public:
 void addToy(std::unique_ptr<Toy> toy) {
 toys.push_back(std::move(toy));
 // No modification needed to add new toys!
 }
};
```

**In Our Notification Example:** The NotificationManager can add new types of senders without changing its own code.

```

#include <memory>
#include <vector>

// Notification manager - Open for extension, closed for modification
class NotificationManager {
private:
 std::vector<std::unique_ptr<INotificationSender>> senders; // List of senders

public:
 // You can add ANY new sender without changing this class!
 void addSender(std::unique_ptr<INotificationSender> sender) {
 senders.push_back(std::move(sender));
 }

 // Send to ALL registered senders
 void sendAll(const std::string& message, const std::string& recipient) {
 for (const auto& sender : senders) {
 sender->send(message, recipient);
 }
 }

 // Send to a specific type of sender
 void sendToType(const std::string& type, const std::string& message, const std::string& recipient) {
 for (const auto& sender : senders) {
 if (getSenderType(sender.get()) == type) {
 sender->send(message, recipient);
 break;
 }
 }
 }

private:
 // Helper to identify sender types
 std::string getSenderType(const INotificationSender* sender) const {
 if (dynamic_cast<const EmailNotificationSender*>(sender)) return "email";
 if (dynamic_cast<const SmsNotificationSender*>(sender)) return "sms";
 if (dynamic_cast<const PushNotificationSender*>(sender)) return "push";
 return "unknown";
 }
};

// Example usage:
int main() {
 NotificationManager manager;

 // Add existing senders
 manager.addSender(std::make_unique<EmailNotificationSender>());
 manager.addSender(std::make_unique<SmsNotificationSender>());

 // Want to add WhatsApp notifications? Just create WhatsAppNotificationSender
 // and add it - NO CHANGES to NotificationManager needed!
 // manager.addSender(std::make_unique<WhatsAppNotificationSender>());

 // Send to all
 manager.sendAll("Hello!", "user@example.com");

 // Send only email
 manager.sendToType("email", "Welcome!", "user@example.com");

 return 0;
}

```

#### Why OCP is Important:

- **Easy to add features:** Want Slack notifications? Just create SlackNotificationSender and add it

- **No breaking changes:** Adding new features doesn't break existing code
- **Future-proof:** Your code can grow without constant rewrites
- **Team friendly:** Multiple developers can add features without stepping on each other's code

### Step 3: Liskov Substitution Principle (LSP) - "Children Should Act Like Their Parents"

**\*\*Imagine:\*\*** You have a "Bird" class that can fly. If you create a "Penguin" that inherits from Bird, but penguins can't fly, that's

**\*\*The Problem (Bad Code):\*\***

```
```cpp
class Bird {
public:
    virtual void fly() { std::cout << "Flying!" << std::endl; }
};

class Penguin : public Bird {
public:
    void fly() override {
        throw std::runtime_error("Penguins can't fly!");
    }
};

void makeBirdFly(Bird* bird) {
    bird->fly(); // Crashes if penguin!
}
```

The Solution (Good Code):

```
class Bird {
public:
    virtual void move() { std::cout << "Moving!" << std::endl; }
};

class Sparrow : public Bird {
public:
    void fly() override { std::cout << "Flying high!" << std::endl; }
    void move() override { fly(); }
};

class Penguin : public Bird {
public:
    void swim() { std::cout << "Swimming!" << std::endl; }
    void move() override { swim(); }
};
```

In Our Notification Example: All notification senders must actually send notifications to the recipient.

```

// All concrete senders properly implement INotificationSender
// This ensures LSP compliance - any INotificationSender can be used wherever INotificationSender is expected

// Example of LSP violation (don't do this):
class BrokenPushSender : public INotificationSender {
public:
    void send(const std::string& message, const std::string& recipient) override {
        // This violates LSP because it doesn't actually send to the recipient
        // It just logs and doesn't perform the expected behavior
        std::cout << "Logging message: " << message << " (not sending to " << recipient << ")" << std::endl;
    }
};

// Correct implementation - actually sends to the recipient
class WorkingPushSender : public INotificationSender {
public:
    void send(const std::string& message, const std::string& recipient) override {
        // Validate and actually send to the recipient
        validateDeviceToken(recipient);
        std::cout << "Sending push notification to " << recipient << ": " << message << std::endl;
        // Connect to push service and deliver to device
    }

private:
    void validateDeviceToken(const std::string& token) {
        if (token.length() < 10) {
            throw std::invalid_argument("Invalid device token");
        }
    }
};

// This function can work with ANY INotificationSender
void sendNotification(INotificationSender* sender, const std::string& message, const std::string& recipient) {
    sender->send(message, recipient); // Will work the same way for all senders
}

int main() {
    WorkingPushSender workingSender;
    // BrokenPushSender brokenSender; // Don't use this!

    // Both should work identically
    sendNotification(&workingSender, "Hello!", "device123");
    // sendNotification(&brokenSender, "Hello!", "device123"); // Would not actually send!

    return 0;
}

```

Why LSP is Important:

- **Reliable code:** You can trust that any `INotificationSender` will actually send notifications
- **Easy testing:** Test with any concrete sender and expect the same behavior
- **Polymorphism works:** Parent class pointers can be used safely with child objects
- **No surprises:** Code behaves as expected, no hidden gotchas

Step 4: Interface Segregation Principle (ISP) - "Don't Force Unwanted Methods on Classes"

****Imagine:**** You have a universal remote control with buttons for TV, DVD, and radio. But your TV doesn't need the radio buttons, and

****The Problem (Bad Code):****

```
```cpp
class UniversalRemote {
public:
 virtual void turnOnTV() = 0;
 virtual void turnOnDVD() = 0;
 virtual void turnOnRadio() = 0;
 // TV class is forced to implement radio methods it doesn't need!
};

class TV : public UniversalRemote {
public:
 void turnOnTV() override { /* TV logic */ }
 void turnOnDVD() override { /* ??? TV doesn't control DVD */ }
 void turnOnRadio() override { /* ??? TV doesn't control radio */ }
};
```

**The Solution (Good Code):**

```
class ITVControl {
public:
 virtual void turnOnTV() = 0;
};

class IDVDControl {
public:
 virtual void turnOnDVD() = 0;
};

class TV : public ITVControl {
public:
 void turnOnTV() override { /* only TV logic */ }
};

class DVD : public IDVDControl {
public:
 void turnOnDVD() override { /* only DVD logic */ }
};
```

**In Our Notification Example:** Different senders implement only the interfaces they need.



```

#include <vector>

// Basic sending interface
class INotificationSender {
public:
 virtual void send(const std::string& message, const std::string& recipient) = 0;
 virtual ~INotificationSender() = default;
};

// Separate interface for senders that can retry failed sends
class IRetryableSender {
public:
 virtual bool retry(const std::string& message, const std::string& recipient, int maxRetries) = 0;
 virtual ~IRetryableSender() = default;
};

// Separate interface for senders that can send to multiple recipients at once
class IBulkSender {
public:
 virtual void sendBulk(const std::vector<std::string>& messages, const std::vector<std::string>& recipients) = 0;
 virtual ~IBulkSender() = default;
};

// Email sender with retry capability - implements both interfaces it needs
class RetryableEmailSender : public EmailNotificationSender, public IRetryableSender {
public:
 bool retry(const std::string& message, const std::string& recipient, int maxRetries) override {
 for (int i = 0; i < maxRetries; ++i) {
 try {
 send(message, recipient); // Uses inherited send method
 return true;
 } catch (const std::exception& e) {
 std::cout << "Retry " << (i + 1) << " failed: " << e.what() << std::endl;
 }
 }
 return false;
 }
};

// SMS sender with bulk capability - implements both interfaces it needs
class BulkSmsSender : public SmsNotificationSender, public IBulkSender {
public:
 void sendBulk(const std::vector<std::string>& messages, const std::vector<std::string>& recipients) override {
 if (messages.size() != recipients.size()) {
 throw std::invalid_argument("Messages and recipients count must match");
 }
 for (size_t i = 0; i < messages.size(); ++i) {
 send(messages[i], recipients[i]); // Uses inherited send method
 }
 }
};

// Basic push sender - only implements basic sending, no extra interfaces
class BasicPushSender : public PushNotificationSender {
 // No extra methods - just basic sending
};

// Usage example:
void sendWithRetry(IRetryableSender* sender, const std::string& message, const std::string& recipient) {
 if (!sender->retry(message, recipient, 3)) {
 std::cout << "Failed to send after retries" << std::endl;
 }
}

void sendBulk(IBulkSender* sender, const std::vector<std::string>& messages, const std::vector<std::string>& recipients) {

```

```

 sender->sendBulk(messages, recipients);
}

int main() {
 RetryableEmailSender emailSender;
 BulkSmsSender smsSender;
 BasicPushSender pushSender;

 // Use only the interfaces each sender supports
 sendWithRetry(&emailSender, "Hello!", "user@example.com");
 sendBulk(&smsSender, {"Msg1", "Msg2"}, {"+123", "+456"});

 // pushSender doesn't support retry or bulk, so we don't force it to
 INotificationSender* basicSender = &pushSender;
 basicSender->send("Basic message", "device123");

 return 0;
}

```

#### Why ISP is Important:

- **Clean interfaces:** Classes only implement what they actually need
- **Less coupling:** Changes to retry logic don't affect basic senders
- **Easier testing:** Test only the methods each class actually has
- **More flexible:** Can add new capabilities without changing existing classes

### Step 5: Dependency Inversion Principle (DIP) - "Depend on Ideas, Not Specific Things"

**\*\*Imagine:\*\*** You want to drink coffee. Instead of depending on a specific coffee machine, you depend on the idea of "something that makes coffee"

**\*\*The Problem (Bad Code):\*\***

```

```cpp
class CoffeeDrinker {
private:
    SpecificCoffeeMachine machine; // Depends on specific machine
public:
    void drink() {
        machine.makeCoffee(); // Stuck with this machine forever!
    }
};

```

The Solution (Good Code):

```

class IHotDrinkMaker {
public:
    virtual void makeDrink() = 0;
};

class CoffeeDrinker {
private:
    std::unique_ptr<IHotDrinkMaker> drinkMaker; // Depends on idea
public:
    CoffeeDrinker(std::unique_ptr<IHotDrinkMaker> maker) : drinkMaker(std::move(maker)) {}
    void drink() {
        drinkMaker->makeDrink(); // Can use any drink maker!
    }
};

```

In Our Notification Example: High-level business logic depends on abstractions, not specific sender implementations.

```

// High-level business logic depends on abstraction, not concrete implementations
class NotificationService {
private:
    std::unique_ptr<NotificationManager> manager; // Depends on abstraction

public:
    // Constructor takes abstraction (interface), not concrete class
    NotificationService(std::unique_ptr<NotificationManager> mgr) : manager(std::move(mgr)) {}

    // Business logic: sending welcome notifications
    void sendWelcomeNotification(const std::string& userId) {
        std::string message = "Welcome to our platform!";
        std::string recipient = getUserContact(userId);
        // Uses abstraction - doesn't know which senders are actually used
        manager->sendToType("email", message, recipient);
    }

    // Business logic: sending security alerts
    void sendSecurityAlert(const std::string& userId) {
        std::string message = "Security alert: Unusual activity detected";
        std::string recipient = getUserContact(userId);
        // Uses abstraction - can easily switch from SMS to email or push
        manager->sendToType("sms", message, recipient);
    }

private:
    std::string getUserContact(const std::string& userId) {
        // Simulate database lookup
        return "user" + userId + "@example.com";
    }
};

// Factory creates the specific implementations, but business logic doesn't know about them
class NotificationServiceFactory {
public:
    // Basic service with standard senders
    static std::unique_ptr<NotificationService> createService() {
        auto manager = std::make_unique<NotificationManager>();
        manager->addSender(std::make_unique<EmailNotificationSender>());
        manager->addSender(std::make_unique<SmsNotificationSender>());
        manager->addSender(std::make_unique<PushNotificationSender>());

        return std::make_unique<NotificationService>(std::move(manager));
    }

    // Advanced service with retry and bulk capabilities
    static std::unique_ptr<NotificationService> createAdvancedService() {
        auto manager = std::make_unique<NotificationManager>();
        manager->addSender(std::make_unique<RetryableEmailSender>());
        manager->addSender(std::make_unique<BulkSmsSender>());
        manager->addSender(std::make_unique<PushNotificationSender>());

        return std::make_unique<NotificationService>(std::move(manager));
    }

    // Test service with mock senders (doesn't actually send anything)
    static std::unique_ptr<NotificationService> createTestService() {
        auto manager = std::make_unique<NotificationManager>();
        // Add mock senders for testing
        // manager->addSender(std::make_unique<MockEmailSender>());
        // manager->addSender(std::make_unique<MockSmsSender>());

        return std::make_unique<NotificationService>(std::move(manager));
    }
};

```

```

int main() {
    // Business logic doesn't know or care which senders are used!
    auto service = NotificationServiceFactory::createService();

    // Same code works regardless of sender implementations
    service->sendWelcomeNotification("user123");
    service->sendSecurityAlert("user456");

    // Can easily switch to advanced service without changing business logic
    auto advancedService = NotificationServiceFactory::createAdvancedService();
    advancedService->sendWelcomeNotification("user789");

    // Can use test service for development without changing any business code
    // auto testService = NotificationServiceFactory::createTestService();

    return 0;
}

```

Why DIP is Important:

- **Easy to change:** Want to use database instead of files? Just change the factory
- **Easy to test:** Inject mock objects without changing business logic
- **Flexible:** Business rules don't get tied to specific technologies
- **Reusable:** Business logic can work with different implementations

```

## Complete Example

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <memory>

// Include all the classes defined above...

int main() {
 try {
 // Create notification service using factory
 auto service = NotificationServiceFactory::createService();

 // Send different types of notifications
 service->sendWelcomeNotification("123");
 service->sendSecurityAlert("456");

 // Demonstrate OCP - we can add new senders without modifying existing code
 auto manager = std::make_unique<NotificationManager>();
 manager->addSender(std::make_unique<EmailNotificationSender>());
 manager->addSender(std::make_unique<SmsNotificationSender>());
 manager->addSender(std::make_unique<PushNotificationSender>());

 // Send to all registered senders
 manager->sendAll("System maintenance tonight", "all_users");

 } catch (const std::exception& e) {
 std::cerr << "Error: " << e.what() << std::endl;
 return 1;
 }

 return 0;
}

```

## Why SOLID Makes Your Code Better (Like a Superhero Team!)

Imagine your code is a superhero team:

- **SRP:** Each superhero has one special power (not trying to do everything)
- **OCP:** The team can add new heroes without rebuilding the headquarters
- **LSP:** New heroes can fill in for old ones without breaking the mission
- **ISP:** Each hero only carries the gadgets they actually use
- **DIP:** The team depends on the "idea" of saving people, not specific tools

## Benefits of SOLID Design (Real-World Advantages)

1. **Easier to Fix Bugs:** When something breaks, you know exactly which class to look at
2. **Add New Features Without Breaking Old Ones:** Like adding a new notification type without crashing existing code
3. **Test Individual Pieces:** You can test email sending without worrying about SMS code
4. **Change Your Mind Easily:** Want to switch from SMS to WhatsApp? Just swap the sender class
5. **Reuse Code:** Use your EmailNotificationSender in completely different projects

## Testing the SOLID Design (Making Sure It Works)

```
// Example unit test (simplified) - like checking if each superhero can do their job
void testEmailSender() {
 EmailNotificationSender sender;

 // Test normal case
 sender.send("Test message", "test@example.com");
 // Should work without errors

 // Test error case
 try {
 sender.send("Test", "invalid-email"); // Bad email
 // If we get here, test failed!
 assert(false);
 } catch (const std::invalid_argument&) {
 // Good! It caught the error as expected
 }
}

void testNotificationManager() {
 NotificationManager manager;
 manager.addSender(std::make_unique<EmailNotificationSender>());
 manager.addSender(std::make_unique<SmsNotificationSender>());

 // Test sending to specific types
 manager.sendToType("email", "Test", "user@example.com");
 manager.sendToType("sms", "Test", "+1234567890");
 // Should work without mixing up email and SMS
}
```

## Final Thoughts: SOLID is Like Building with LEGO

SOLID principles help you build code that's:

- **Easy to understand** (each piece has one job)
- **Easy to change** (add new pieces without breaking old ones)
- **Reliable** (pieces work predictably)
- **Reusable** (use pieces in different projects)

Start with small projects and apply one principle at a time. Soon you'll wonder how you ever coded without SOLID!

Remember: Good code is like a well-organized kitchen - everything has its place, and you can find what you need quickly!